

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

**ACTIVIDAD LABORATORIO NO.1
PROGRAMACIÓN DE SCRIPTS EN LINUX**

**PRESENTADO POR:
ALEJANDRO DE MENDOZA**

**PRESENTADO AL PROFESOR:
ING LAERTE ALBERTO FERREIRA CASTRO**

**FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
BOGOTÁ D.C.
26 DE MAYO
2025**

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

TABLA DE CONTENIDO

<i>Instalación de VirtualBox.....</i>	<i>4</i>
<i>Descargar una imagen ISO de Linux</i>	<i>4</i>
<i>Crear una nueva máquina virtual en VirtualBox.....</i>	<i>4</i>
Iniciar VM:	7
<i>Desarrollo de la actividad propuesta en el laboratorio.....</i>	<i>8</i>
<i>Comandos en Linux.....</i>	<i>8</i>
1. Comando \$ PWD:	8
2. Comando \$ mkdir	9
3. Comando \$ man	9
4. Comando \$ cd	10
5. Comando \$ ls.....	10
6. Comando \$ top	11
7. Comando \$ apt-get.....	11
8. Comando \$ cat	11
9. Comando \$ who	12
10. Comando \$ rmdir	12
11. Comando \$ ifconfig	13
12. Comando \$ date	13
13. Comando \$ clear	13
14. Comando \$ poweroff	14
15. Comando \$ shutdown	14
<i>Scripts en Linux.....</i>	<i>15</i>
1. Script ¡Hola Mundo!	15
2. Script Mostrar Parámetros	15
3. Script VerificaParametros	16
4. Script ParametrosComas.sh	17
5. Script ParametrosComasAyuda.Sh	17
6. Script UsuariosConectados.sh	18
7. Script usuarioconectado	19
8. Script modificar_path.sh	20
9. Script modificar_script_para_llamar_usuarioconectado.sh	20
<i>Resumen de los scripts</i>	<i>21</i>
<i>CONCLUSIÓN DE LA ACTIVIDAD</i>	<i>22</i>
<i>BIBLIOGRAFÍA</i>	<i>22</i>

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

RESUMEN

Este laboratorio me permitió fortalecer mis competencias en el uso de máquinas virtuales, administración de sistemas y configuración de entornos Linux, especialmente enfocados en la instalación y prueba de sistemas operativos en entornos controlados. Para ello, trabajé con las herramientas VirtualBox y una imagen ISO de Linux Mint, lo que me permitió explorar conceptos de virtualización y prácticas de sistemas operativos sin comprometer el equipo físico.

Inicié descargando e instalando VirtualBox en mi equipo con Windows, asegurando una base sólida para correr máquinas virtuales. Luego, accedí a la guía oficial de instalación de Linux Mint y seleccioné la edición más adecuada, descargando su respectiva imagen ISO. Esta imagen funcionó como disco de arranque para el sistema invitado.

Posteriormente, creé en VirtualBox una nueva máquina virtual, asignando recursos como memoria RAM, disco duro virtual y procesador. Monté la ISO de Linux Mint en el entorno virtual y procedí con el arranque, el cual me llevó al entorno Live Boot, desde donde inicié el proceso de instalación. Este proceso incluyó la selección de idioma, configuración regional, creación de usuario y el particionado automático del disco virtual.

Durante la instalación, me familiaricé con el entorno de escritorio de Linux Mint y con los conceptos básicos del sistema de archivos, permisos y usuarios. Esta práctica resultó fundamental para comprender cómo se estructura y administra un sistema Linux real.

Finalmente, completada la instalación, pude acceder a un entorno Linux funcional desde VirtualBox, donde probé comandos básicos como ls, cd, mkdir, y verifiqué el funcionamiento general del sistema. Esta experiencia me permitió visualizar de forma práctica el uso de un sistema operativo alternativo, validar conocimientos teóricos y preparar el entorno para futuras actividades relacionadas con administración de procesos, memoria, archivos y usuarios.

Este laboratorio no solo me permitió instalar y configurar Linux Mint, sino también integrar conceptos clave de virtualización, sistemas operativos y administración de entornos, destacando la importancia de herramientas como VirtualBox para el aprendizaje seguro y controlado de tecnologías de software libre.

INTRODUCCIÓN

En este laboratorio se busca fortalecer las competencias en la virtualización de sistemas operativos, así como en la instalación, configuración y administración básica de entornos Linux.

Ahora mediante el uso de VirtualBox y la distribución Linux Mint, se implementa una máquina virtual funcional, lo cual va a brindar una experiencia práctica sobre el despliegue de sistemas en un entorno seguro y aislado del sistema anfitrión.

El objetivo principal es simular un entorno real de trabajo en Linux sin necesidad de modificar o arriesgar el sistema operativo principal del equipo. Esta experiencia es clave para explorar conceptos fundamentales de los sistemas operativos avanzados, como la gestión de recursos, la estructura del sistema de archivos, y el uso de comandos básicos de administración.

Gracias al uso de una imagen ISO de Linux Mint y su ejecución en VirtualBox, se va a comprender de manera visual y aplicada el proceso de instalación de un sistema operativo desde cero, incluyendo la asignación de hardware virtual (RAM, CPU y disco), el montaje de unidades y el arranque del sistema. Esta práctica inicial se convierte en la base para

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

profundizar en temas como la gestión de procesos, memoria, dispositivos de entrada/salida y servicios del sistema.

En resumen, este laboratorio es fundamental para introducir un entorno Linux de manera práctica y controlada, permitiendo afianzar conocimientos teóricos sobre sistemas operativos y sentar las bases para realizar tareas de mayor complejidad. A continuación, el desarrollo.

DESARROLLO ACTIVIDAD

El presente laboratorio tiene como objetivo poner en práctica los conocimientos adquiridos sobre el uso del intérprete de comandos en Linux y la programación de scripts en Bash. A través de la creación y ejecución de scripts sencillos, se busca reforzar la comprensión de los comandos básicos del sistema, el manejo de estructuras de control, variables y funciones, fundamentales para la automatización de tareas administrativas en entornos Unix/Linux. A continuación, procedo con el desarrollo del entorno para luego proceder con la explicación de detallada de los comandos y scripts en Linux.

Desarrollo Del Entorno

Instalación de VirtualBox

Para iniciar este laboratorio lo primero en mi caso fue generar la instalación de la herramienta VirtualBox con base en las instrucciones brindadas por el profesor en la clase, y para esto me dirigí al siguiente link:

- <https://www.virtualbox.org/wiki/Downloads>

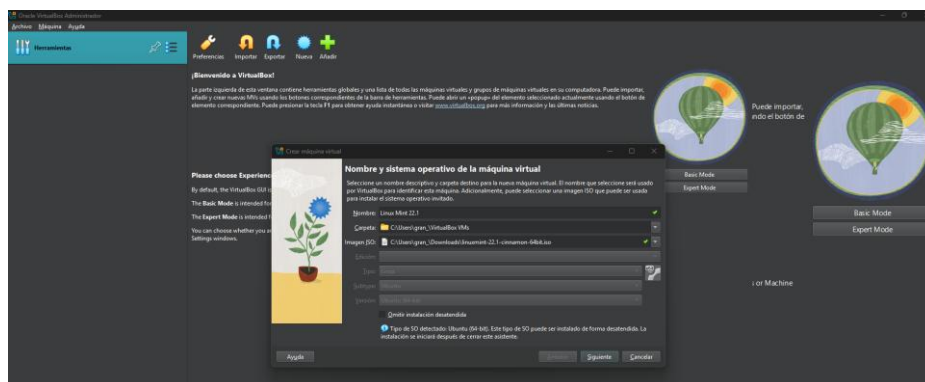
Descargar una imagen ISO de Linux

Luego entonces procedí a descargar la imagen ISO de Linux en mi caso de Linux Mint en el siguiente link:

- <https://linuxmint.com/download.php>

Crear una nueva máquina virtual en VirtualBox

Una vez de mi parte habiendo descargado la herramienta de VirtualBox y la ISO de Linux Mint procedí a crear una máquina virtual, entonces para esto lo primero es abrir VirtualBox y dar clic en nueva y nos aparece la siguiente ventana:

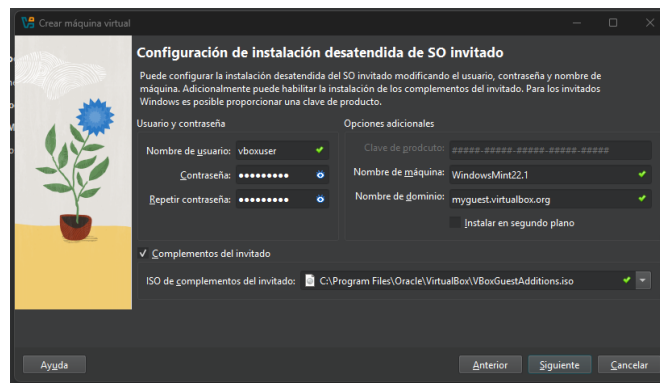


Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

En donde como podemos ver coloque el nombre de la maquina como “Linux Mint 22.1”, la ubicación de esta es en C:\Users\gran_\VirtualBox VMs y la iso descargada recibe el nombre de linuxmint-22.1-cinnamon-64bit.iso y su ubicación es:

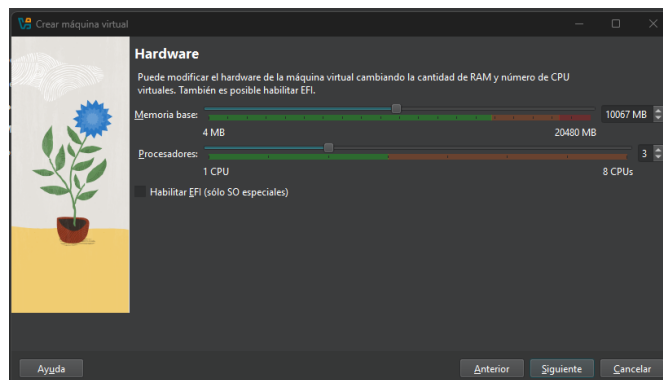
- C:\Users\gran_\Downloads\linuxmint-22.1-cinnamon-64bit.iso

Luego de esto al dar clic en siguiente aparece la siguiente pantalla:



La cual como podemos ver deje el nombre de usuario por defecto que es vboxuser, agregue una contraseña, le di el nombre a la maquina como LinuxMint22.1 y di check en complementos del invitado ya que son herramientas adicionales de VirtualBox que se instalan automáticamente y que mejoran la integración con el sistema operativo invitado (como mejor resolución, arrastrar archivos, etc.).

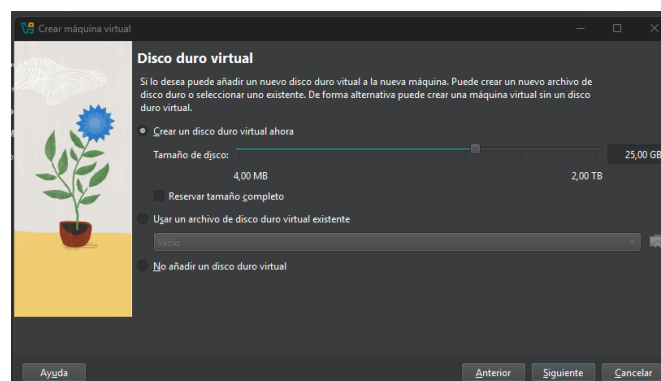
Al dar clic entonces en siguiente aparece la siguiente ventana:



En donde asigno de memoria base (RAM) 10067 MB y 3 procesadores.

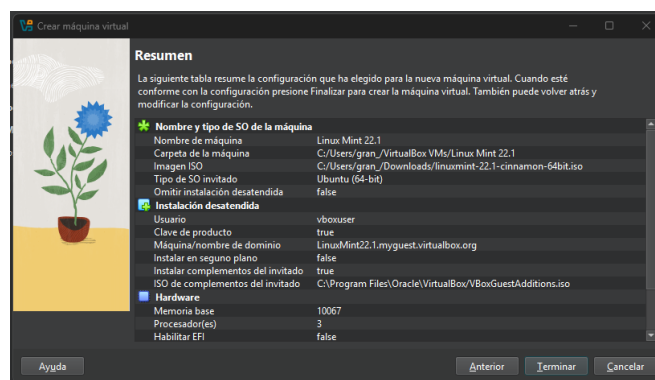
Ahora al dar siguiente aparece la siguiente pantalla:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

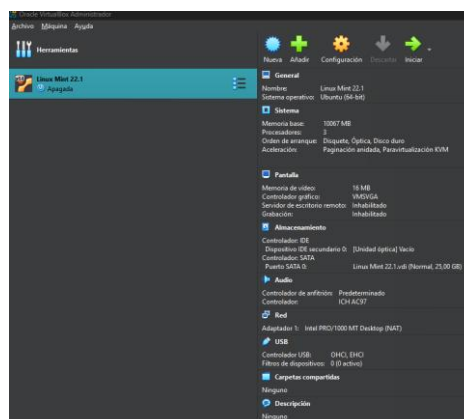


Y en este caso selecciono crear un disco duro virtual ahora ya que es lo que necesito para crear un nuevo disco virtual donde se instalará Linux Mint. Por otra parte, dejo un tamaño de disco: 25.00 GB ya que considero este es un valor adecuado para una instalación básica de Linux Mint con espacio suficiente para actualizaciones y software adicional. Por otra parte, no marco reservar tamaño completo ya que espero que el disco se expanda según el uso, ocupando menos espacio en mi disco real al inicio. No tengo un disco virtual creado previamente, lo cual no aplica en este caso. Y por último tampoco marco no añadir un disco duro virtual, porque sin disco duro no puedo instalar el sistema operativo.

Entonces al dar clic en siguiente aparece la siguiente ventana:



Y acá simplemente doy clic en terminar y me queda mi máquina virtual de Linux Mint creada.



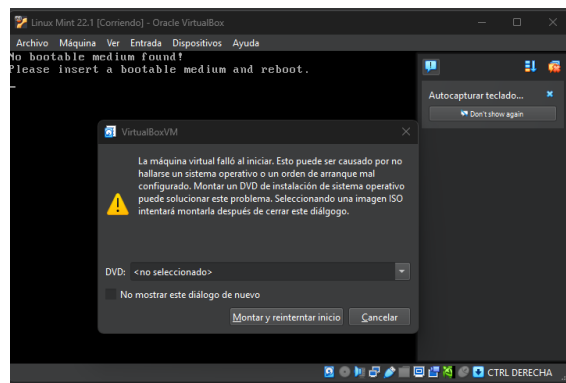
Y entonces me aparecen las siguientes características de la MV:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

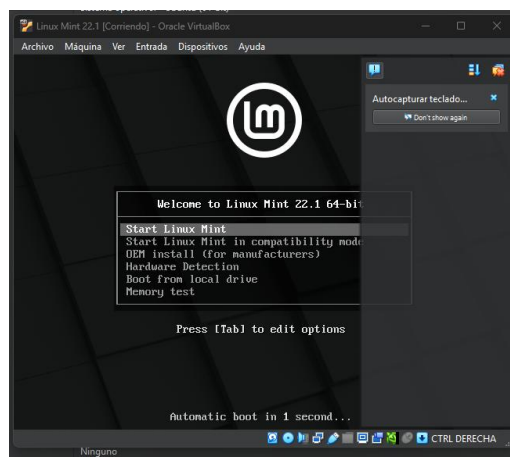
- Nombre: Linux Mint 22.1
- Sistema operativo: Ubuntu (64-bit) → Porque Linux Mint se basa en Ubuntu.
- Estado: Apagada → Aún no he iniciado la instalación.
- Memoria base: 10067 MB (≈10 GB) → Excelente para un rendimiento fluido.
- Procesadores: 3 porque mi equipo tiene al menos 4 núcleos físicos.
- Orden de arranque: Disquete, Óptica, Disco duro
- Aceleración: Activada → Mejora el rendimiento.
- Memoria de vídeo: 16 MB y se puede aumentar a 128 MB para un mejor soporte gráfico.
- Controlador gráfico: VMSVGA → Es el recomendado para sistemas Linux modernos.
- Unidad óptica (IDE): Vacía → Aquí monto la imagen ISO de Linux Mint.
- Disco duro (SATA): Linux Mint 22.1.vdi (25.00 GB).
- Adaptador 1: Intel PRO/1000 MT Desktop (NAT) → Correcto para tener acceso a internet desde la VM.

Iniciar VM:

Ahora entonces selecciono mi máquina virtual y le doy iniciar y aparece esta ventana:

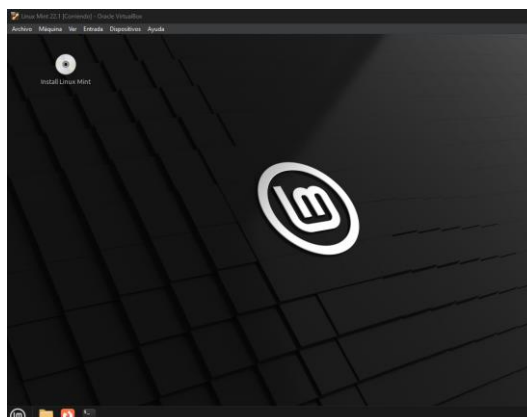


Entonces acá le doy clic en no mostrar dialogo de nuevo y selecciono en DVD la imagen ISO que descargue y le doy clic en Montar y reiniciar inicio y listo.



Acá doy clic en Start Linux Mint y espero un poco y me aparece la siguiente imagen:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	



Y acá entonces le doy instalar Linux Mint, selecciono el idioma y la disposición del teclado, le doy check en instalar los códecs, selecciono borrar disco e instalar Windows Mint, luego continuar, ´le doy Bogotá en mi ubicación, selecciono un usuario y una contraseña, doy clic en continuar arranca la instalación y al final de todo, luego de un reinicio aparece la siguiente ventana:



Y ahora procedo a configurar Linux a mi antojo, es importante indicar que yo en mi caso active los Guest Additions y adicionalmente actualice el SO.

Desarrollo de la actividad propuesta en el laboratorio

Bueno como primera medida es importante indicar que este laboratorio se basa en la programación de scripts sencillos y el uso de comandos del sistema. Por lo que de mi parte primero se ejecutarán todos los comandos para luego dar a conocer los scripts y de esta manera dar un orden al contexto de este laboratorio:

Comandos en Linux

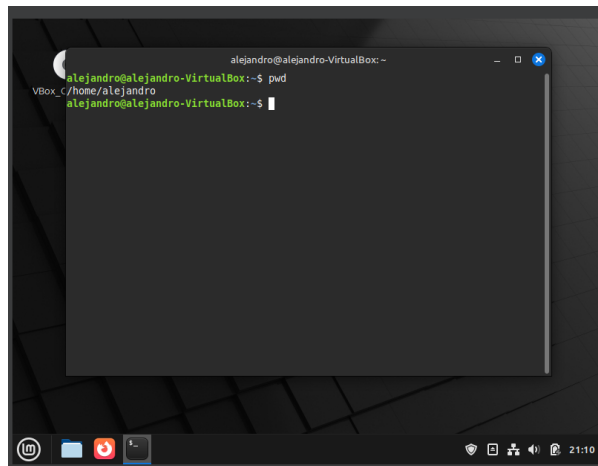
Para iniciar este desarrollo debo indicar que un comando en Linux es un programa o una orden que se ejecuta a través del intérprete de comandos (generalmente bash) para realizar acciones como copiar archivos, crear directorios, listar contenido, gestionar procesos, configurar permisos, entre muchas otras tareas. A continuación, la lista de los vistos en clase, objeto del desarrollo de este laboratorio:

1. Comando \$ PWD:

Print Working Directory: (Imprimir el directorio de trabajo actual), Me dice en qué carpeta (directorio) estoy ubicado actualmente dentro del sistema de archivos. Es muy útil cuando

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

estoy navegando entre carpetas con `cd` y quiero confirmar mi ubicación. Entonces en Linux abro una terminal y ejecuto el comando:



```

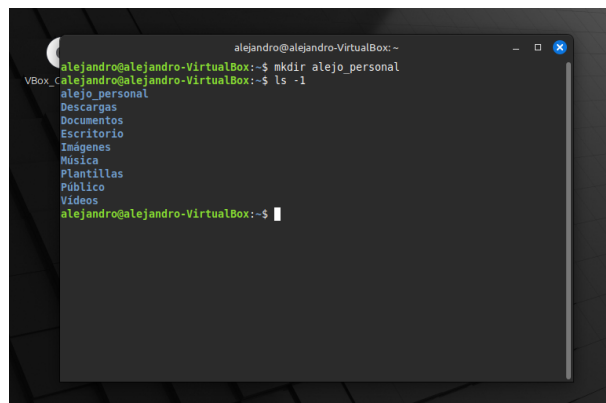
alejandro@alejandro-VirtualBox: ~
alejandro@alejandro-VirtualBox:~$ pwd
VBox_/_home/alejandro
alejandro@alejandro-VirtualBox:~$

```

Y como podemos ver me muestra el directorio actual.

2. Comando \$ mkdir

Ahora utilizamos el comando `mkdir`, y este comando sirve para crear un directorio (carpeta). Entonces coloco `mkdir alejo_personal` y me crea la carpeta `alejo_personal`, a continuación la respectiva imagen:



```

alejandro@alejandro-VirtualBox: ~
alejandro@alejandro-VirtualBox:~$ mkdir alejo_personal
VBox_/_alejandro@alejandro-VirtualBox:~$ ls -l
alejo_personal
Descargas
Documentos
Escritorio
Imágenes
Música
Plantillas
Público
Videos
alejandro@alejandro-VirtualBox:~$

```

3. Comando \$ man

El siguiente comando es `man`, y muestra una página de manual que incluye una descripción, opciones, ejemplos y, a veces, información adicional sobre el comando especificado.

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

```

alejandro@alejandro-VirtualBox: ~
alejandro@alejandro-VirtualBox:~$ man ls
VBox_C_Qué página de manual quiere?
Por ejemplo, pruebe 'man man'.
alejandro@alejandro-VirtualBox:~$ man ls
alejandro@alejandro-VirtualBox:~$

```

En este caso como se puede denotar utilice “man ls” y me mostro la siguiente información:

```

LS(1)                                User Commands                                LS(1)
NAME
ls - list directory contents

SYNOPSIS
ls [OPTION]... [FILE]...

DESCRIPTION
List information about the FILES (the current directory by default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is speci-
fied.

Mandatory arguments to long options are mandatory for short options
too.

-a, --all
do not ignore entries starting with .
-A, --almost-all
do not list implied . and ..

--author
Manual page ls(1) line 1 (press h for help or q to quit)

```

4. Comando \$ cd

Este comando sirve para cambiar de directorio, es decir, para moverse entre carpetas en la terminal. En este caso utilice cd Documentos para ir a la carpeta de documentos.

```

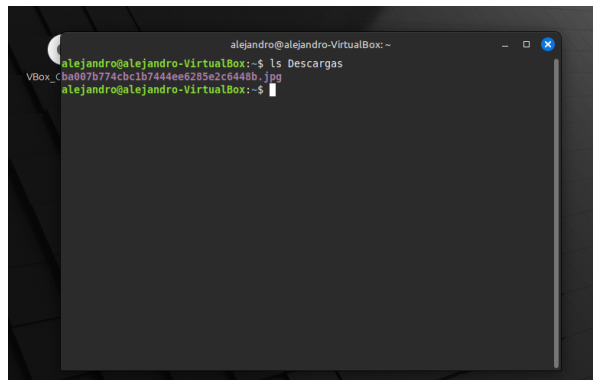
alejandro@alejandro-VirtualBox: ~
alejandro@alejandro-VirtualBox:~$ cd Documentos
VBox_C_alejandro@alejandro-VirtualBox:~/Documentos$

```

5. Comando \$ ls

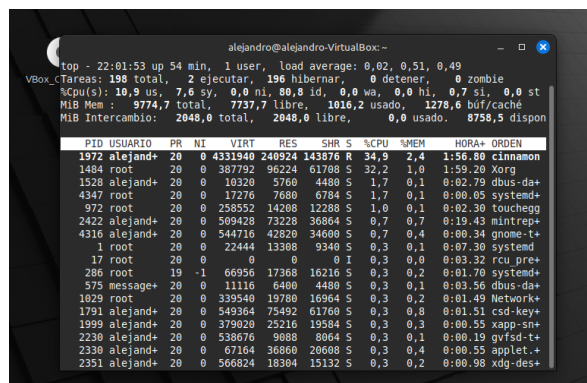
Este comando se usa para listar el contenido de un directorio, es decir, muestra los archivos y carpetas que hay dentro, en este caso como ejemplo tome ls Descargas donde descargue una imagen de Pinterest, a continuación, la imagen:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	



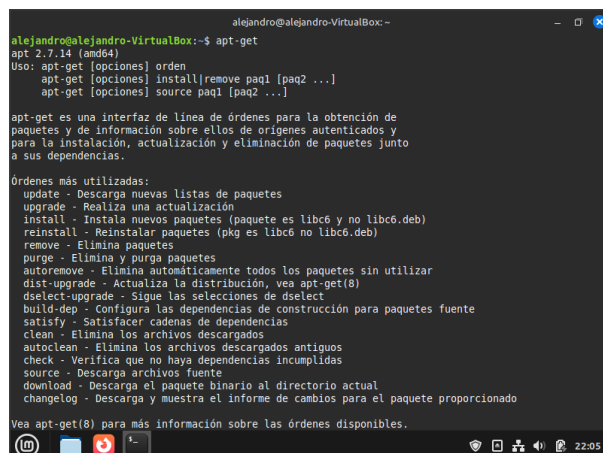
6. Comando \$ top

Sirve para monitorear en tiempo real los procesos del sistema. Muestra una lista dinámica de los procesos que están usando recursos del sistema, como CPU y memoria.



7. Comando \$ apt-get

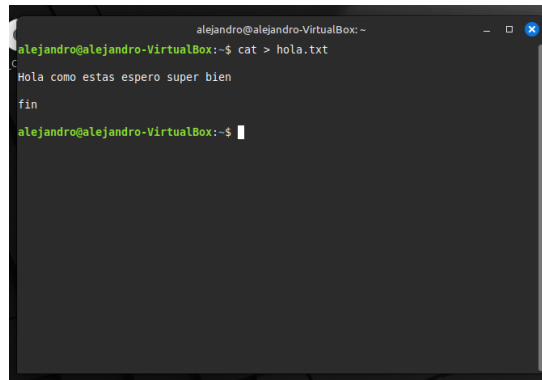
Este comando generalmente usado en distribuciones basadas en Debian como Ubuntu, sirve para gestionar paquetes desde la línea de comandos. Con él puedes instalar, actualizar y eliminar software del sistema. A continuación, la respectiva imagen:



8. Comando \$ cat

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

Este comando se usa para ver, concatenar y crear archivos de texto desde la terminal. En este caso yo procedí a crear un nuevo archivo de nombre hola.txt, a continuación, la respectiva imagen:



```

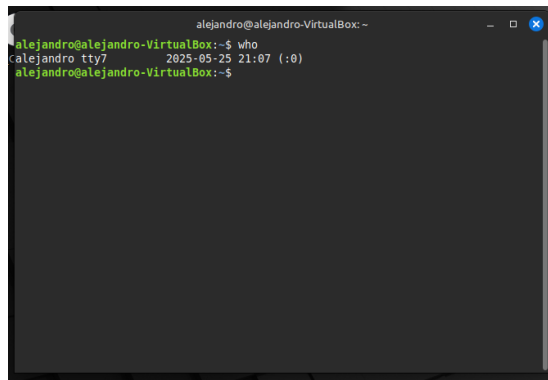
alejandro@alejandro-VirtualBox: ~
alejandro@alejandro-VirtualBox:~$ cat > hola.txt
c
Hola como estas espero super bien
fin
alejandro@alejandro-VirtualBox:~$

```

Nota es importante indicar que luego de escribir el texto se presiona Ctrl + D para guardar y salir.

9. Comando \$ who

Este comando se usa para mostrar qué usuarios están conectados al sistema en ese momento. En este caso es claro que único usuario conectado soy yo y por eso me muestra, a continuación, la respectiva imagen de la consola:



```

alejandro@alejandro-VirtualBox: ~
alejandro@alejandro-VirtualBox:~$ who
alejandro tty7 2025-05-25 21:07 (:0)
alejandro@alejandro-VirtualBox:~$

```

10. Comando \$ rmdir

Este comando se usa para se usa para eliminar directorios vacíos. En este caso procedí a eliminar la carpeta de nombre alejo_personal que cree anteriormente y está vacía, a continuación, la respectiva imagen de la consola:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

```

alejandro@alejandro-VirtualBox: ~
alejandro@alejandro-VirtualBox:~$ cat alejo_personal
cat: alejo_personal: Es un directorio
alejandro@alejandro-VirtualBox:~$ rmdir alejo_personal
alejandro@alejandro-VirtualBox:~$ cat alejo_personal
cat: alejo_personal: No existe el archivo o el directorio
alejandro@alejandro-VirtualBox:~$

```

11. Comando \$ ifconfig

Este comando se usa para ver y configurar interfaces de red del sistema, a continuación, la respectiva imagen de la consola:

```

alejandro@alejandro-VirtualBox: ~
alejandro@alejandro-VirtualBox:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fd17:625c:f037:2:6179:fe84:a52e:ed95 prefixlen 64 scopeid 0x0<gl
    obal>
    inet6 fe80::efad:73f4:9305:1f35 prefixlen 64 scopeid 0x20<link>
    inet6 fd17:625c:f037:2:5e87:c8dc:2a33:8982 prefixlen 64 scopeid 0x0<gl
    obal>
    ether 08:00:27:3f:7d:0c txqueuelen 1000 (Ethernet)
    RX packets 24767 bytes 28874871 (28.8 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 6802 bytes 2152401 (2.1 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Bucle local)
    RX packets 966 bytes 113352 (113.3 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 966 bytes 113352 (113.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

alejandro@alejandro-VirtualBox:~$

```

12. Comando \$ date

Este comando se usa para mostrar o establecer la fecha y hora del sistema, a continuación, la respectiva imagen de la consola:

```

alejandro@alejandro-VirtualBox: ~
alejandro@alejandro-VirtualBox:~$ date
dom 25 may 2025 22:27:20 -05
alejandro@alejandro-VirtualBox:~$

```

13. Comando \$ clear

Este comando se usa para limpiar la pantalla de la terminal, es decir, borra todo el contenido visible y deja la terminal "limpia", pero no borra el historial de comandos (puedes seguir viendo los comandos anteriores con las teclas de flecha arriba/abajo). A continuación, la respectiva imagen de la consola:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

The image shows three terminal windows from a VirtualBox environment. The top-left window shows the command 'ls' being executed, listing the contents of the current directory: Descargas, Escritorio, Imágenes, Plantillas, Videos, Documentos, hola.txt, Música, and Público. The top-right window shows the command 'ls' being executed again, displaying the same directory listing. The bottom window shows the command 'ls' being executed, but the output is not visible as the cursor is at the end of the line.

14. Comando \$ poweroff

Este comando se usa para apagar el sistema de forma segura. A continuación, la respectiva imagen de la consola:



15. Comando \$ shutdown

Este comando se usa para apagar o reiniciar el sistema de forma segura y programada, en este caso los programe para que se apague en 30 minutos:

The image shows a terminal window with the following text:


```

alejandro@alejandro-VirtualBox: ~
alejandro@alejandro-VirtualBox:~$ sudo shutdown +30
[sudo] contraseña para alejandro:
Shutdown scheduled for Sun 2025-05-25 23:13:39 -05, use 'shutdown -c' to cancel.
alejandro@alejandro-VirtualBox:~$
    
```

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

Y con este culmino el último de los comandos por desarrollar. Ahora procedo con los respectivos scripts.

Scripts en Linux

Para iniciar esta segunda parte de este laboratorio primero que todo debo indicar que un script en Linux es un conjunto de instrucciones escritas en un lenguaje de comandos (como bash) que se agrupan en un archivo con extensión .sh y se ejecutan en un intérprete de Shell para automatizar tareas. Algunas de sus características principales son:

- Comienza normalmente con la línea **#!/bin/bash** para indicar que se usará el intérprete bash.
- Permite combinar varios comandos del sistema.
- Soporta estructuras de control como condicionales (if, case) y bucles (for, while).
- Puede utilizar variables, funciones, arrays y redireccionamientos.
- Ahorra tiempo y reduce errores al automatizar tareas repetitivas.

Ahora de mi parte considero que los scripts en Linux son esenciales para:

- Automatizar procesos del sistema.
- Realizar tareas administrativas complejas.
- Crear rutinas programadas (cron jobs).
- Desarrollar soluciones personalizadas sin necesidad de software adicional.

A continuación, el desarrollo de los respectivos scripts base de este desarrollo de este laboratorio.

1. Script ¡Hola Mundo!

El script HolaMundo.sh es un ejemplo clásico y minimalista en el mundo de la programación en Linux, diseñado para introducir a los usuarios al uso de scripts en la terminal. Este script consta de una única línea: `echo "Hola Mundo"`, que utiliza el comando `echo` para imprimir el mensaje "Hola Mundo" directamente en la consola. El comando `echo` es una herramienta fundamental en Bash que envía texto a la salida estándar (la terminal), y en este caso, el texto entre comillas dobles se muestra tal cual. Este script no requiere argumentos, no realiza operaciones adicionales ni interactúa con el sistema más allá de mostrar el mensaje, lo que lo convierte en un punto de partida ideal para aprender a crear y ejecutar scripts en Linux. Para ejecutarlo, basta con guardar el código en un archivo con extensión .sh, otorgarle permisos de ejecución con `chmod +x HolaMundo.sh`, y luego correrlo con `./HolaMundo.sh`, lo que mostrará el icónico "Hola Mundo" en la pantalla.

A continuación, la imagen respectiva.

```

alejo@alejo-VirtualBox: ~
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

alejo@alejo-VirtualBox:~$ echo Hola Mundo
Hola Mundo
alejo@alejo-VirtualBox:~$ nano HolaMundo.sh
alejo@alejo-VirtualBox:~$ chmod +x HolaMundo.sh
alejo@alejo-VirtualBox:~$ ./HolaMundo.sh
Hola Mundo
alejo@alejo-VirtualBox:~$

```

2. Script Mostrar Parámetros

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

El script `mostrar_parametros.sh` es una herramienta útil en Linux que utiliza Bash para mostrar los parámetros pasados como argumentos y verificar si corresponden a usuarios conectados, combinando dos funcionalidades prácticas con la ayuda de un script externo. Comienza con `#!/bin/bash` para ejecutarse en Bash y verifica con `if [ $# -eq 0 ]` si se pasaron argumentos; si no, muestra "No se pasaron parámetros", las instrucciones de uso (Uso: \$0 parametro1 [parametro2 ...]), y termina con `exit 1`. Si hay argumentos, imprime "Parámetros recibidos:" y los lista con un guion usando un bucle `for` que recorre "\$@", la variable que contiene todos los argumentos, como se vio al ejecutar `./MuestraParametros.sh hola mundo 123` en la imagen, mostrando "- hola - mundo - 123".

Luego, intenta verificar usuarios conectados con un segundo bucle `for` bajo el encabezado "Comprobando si los usuarios están conectados:", ejecutando `./usuarioconectado` para cada argumento y mostrando el resultado en el formato `usuario → estado (SÍ o NO)`; sin embargo, en la ejecución observada, esta parte falló porque `usuarioconectado` no estaba presente en `/home/alejo`.

Para usar el script correctamente, se crea con `nano` `mostrar_parametros.sh`, se otorgan permisos con `chmod +x`, y se ejecuta en un directorio donde `usuarioconectado` esté disponible, permitiendo listar argumentos y verificar estados de conexión de usuarios en un solo flujo.

A continuación, la imagen respectiva:

```

alejo@alejo-VirtualBox: ~/Ejercicios
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

alejo@alejo-VirtualBox:~$ cd /home/alejo/Ejercicios
alejo@alejo-VirtualBox:~/Ejercicios$ chmod +x mostrar_parametros.sh
alejo@alejo-VirtualBox:~/Ejercicios$ ./mostrar_parametros.sh hola mundo 123
Parámetros recibidos:
- hola
- mundo
- 123

Comprobando si los usuarios están conectados:
hola → NO
mundo → NO
123 → NO
alejo@alejo-VirtualBox:~/Ejercicios$

```

3. Script VerificaParametros

El script `VerificaParametros.sh` es una herramienta sencilla en Linux que utiliza Bash para manejar argumentos pasados desde la terminal, asegurándose de que se proporcione al menos un parámetro y mostrándolos de manera organizada. Comienza con `#!/bin/bash`, indicando que se ejecuta en la shell Bash, y usa `if [ $# -lt 1 ]` para verificar si se pasó al menos un argumento; si no, muestra "Error: Debes introducir al menos un parámetro" junto con las instrucciones de uso (Uso: \$0 parametro1 [parametro2 ...]) y termina con un código de salida 1 para indicar un error.

Cuando se proporcionan argumentos, imprime "Parámetros recibidos:" y los lista uno por uno, precedidos por un guion, mediante un bucle `for` que recorre "\$@", la variable que contiene todos los argumentos pasados; por ejemplo, al ejecutar `./VerificaParametros.sh hola mundo 123` (como se vio en la imagen), muestra "- hola - mundo - 123". Para usar el script, se guarda con extensión `.sh`, se le otorgan permisos con `chmod +x VerificaParametros.sh`, y se ejecuta desde la terminal; si se corre sin argumentos (`./VerificaParametros.sh`), aparece el mensaje de error, y usar `sudo` sin parámetros genera un error adicional, como se observó en la ejecución.

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

Este script es un ejemplo práctico de cómo Bash puede procesar y validar entradas de usuario, siendo útil para tareas administrativas básicas o como base para scripts más complejos.

A continuación, la respectiva imagen:

```

alejo@alejo-VirtualBox: ~
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

alejo@alejo-VirtualBox:~$ nano VerificaParametros.sh
alejo@alejo-VirtualBox:~$ chmod +x VerificaParametros.sh
alejo@alejo-VirtualBox:~$ ./VerificaParametros.sh hola mundo 123
Parámetros recibidos:
- hola
- mundo
- 123
alejo@alejo-VirtualBox:~$ ./VerificaParametros.sh
Error: Debes introducir al menos un parámetro.
Uso: ./VerificaParametros.sh parametro1 [parametro2 ...]
alejo@alejo-VirtualBox:~$

```

4. Script ParametrosComas.sh

El script ParametrosComas.sh es una herramienta práctica en Linux que utiliza Bash para tomar uno o más parámetros ingresados desde la terminal y mostrarlos en una sola línea, separados por comas, ofreciendo una forma ordenada de visualizar argumentos.

Comienza con `#!/bin/bash` para ejecutarse en la shell Bash y verifica con `if [ $# -lt 1 ]` si se pasó al menos un argumento; si no, muestra "Error: Debes introducir al menos un parámetro" junto con las instrucciones de uso (Uso: \$0 parametro1 [parametro2 ...]) y termina con un código de salida 1.

Cuando se proporcionan argumentos, el script construye una cadena en la variable PARAMS usando un bucle for que recorre "\$@", la variable que contiene todos los argumentos: si PARAMS está vacía, se asigna el primer parámetro, y para los siguientes, se añade una coma seguida del argumento, resultando en una salida como "Parámetros recibidos: hola,mundo,123" al ejecutar `./ParametrosComas.sh hola mundo 123`, como se vio en la imagen. Para usarlo, se crea el archivo con `nano ParametrosComas.sh`, se otorgan permisos con `chmod +x ParametrosComas.sh`, y se ejecuta desde la terminal; si no se pasan parámetros, aparecerá el mensaje de error.

Este script, aunque simple, es una base sólida para procesar y formatear entradas de usuario, siendo una versión básica que podría expandirse con funciones adicionales como ayuda detallada.

A continuación, la respectiva imagen:

```

alejo@alejo-VirtualBox: ~
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

alejo@alejo-VirtualBox:~$ nano ParametrosComas.sh
alejo@alejo-VirtualBox:~$ chmod +x ParametrosComas.sh
alejo@alejo-VirtualBox:~$ ./ParametrosComas.sh hola mundo 123
Parámetros recibidos: hola,mundo,123
alejo@alejo-VirtualBox:~$

```

5. Script ParametrosComasAyuda.Sh

El script ParametrosComasAyuda.sh es una herramienta útil en Linux que utiliza Bash para tomar uno o más parámetros ingresados desde la terminal, mostrarlos en una sola línea separados por comas y ofrecer una ayuda detallada en caso de error, mejorando la experiencia del usuario.

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

Comienza con `#!/bin/bash` para ejecutarse en la shell Bash y define una función `mostrar_ayuda()` que proporciona instrucciones de uso (Uso: `$0 parametro1 [parametro2 ...]`), una descripción del script, ejemplos como `$0 hola mundo` (imprime "hola,mundo") o `$0 uno dos tres` (imprime "uno,dos,tres"), y notas sobre la necesidad de al menos un parámetro y la compatibilidad con palabras o números, terminando con `exit 1`. Luego, verifica con `if [ $# -lt 1 ]` si se pasó al menos un argumento; si no, muestra "Error: Debes introducir al menos un parámetro" y llama a `mostrar_ayuda`, como se vio en la ejecución sin parámetros en la imagen. Si hay argumentos, construye una cadena en la variable `PARAMS` usando un bucle `for` que recorre `"$@"`: si `PARAMS` está vacía, toma el primer parámetro, y para los siguientes añade una coma seguida del argumento, resultando en "Parámetros recibidos: hola,mundo,123" al ejecutar `./ParametrosComasAyuda.sh hola mundo 123`, como se mostró. Para usarlo, se crea con `nano ParametrosComasAyuda.sh`, se otorgan permisos con `chmod +x ParametrosComasAyuda.sh`, y se ejecuta desde la terminal; esta versión mejora a scripts similares al incluir ayuda detallada, facilitando su uso para tareas básicas de procesamiento de argumentos.

A continuación, la respectiva imagen:

```

alejo@alejo-VirtualBox: ~
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

alejo@alejo-VirtualBox:~$ nano ParametrosComasAyuda.sh
alejo@alejo-VirtualBox:~$ chmod +x ParametrosComasAyuda.sh
alejo@alejo-VirtualBox:~$ ./ParametrosComasAyuda.sh hola mundo 123
Parámetros recibidos: hola,mundo,123
alejo@alejo-VirtualBox:~$ Parámetros recibidos: hola,mundo,123
Parámetros: no se encontró la orden
alejo@alejo-VirtualBox:~$ Parámetros recibidos:
Parámetros: no se encontró la orden
alejo@alejo-VirtualBox:~$ ./ParametrosComasAyuda.sh
Error: Debes introducir al menos un parámetro.
Uso: ./ParametrosComasAyuda.sh parametro1 [parametro2 ...]

Descripción:
Este script toma uno o más parámetros y los imprime en una sola línea, separados por comas.

Ejemplos:
./ParametrosComasAyuda.sh hola mundo          # Imprime: hola,mundo
./ParametrosComasAyuda.sh uno dos tres        # Imprime: uno,dos,tres

Notas:
- Debes proporcionar al menos un parámetro.
- Los parámetros pueden ser palabras o números.
alejo@alejo-VirtualBox:~$

```

6. Script UsuariosConectados.sh

El script `UsuariosConectados.sh` es una herramienta práctica en Linux que utiliza Bash para verificar si los usuarios pasados como argumentos están conectados al sistema y mostrar los conectados en una sola línea, separados por comas, con una ayuda detallada en caso de error.

Comienza con `#!/bin/bash` para ejecutarse en Bash y define la función `mostrar_ayuda()`, que ofrece instrucciones de uso (Uso: `$0 usuario1 [usuario2 ...]`), una descripción, ejemplos como `$0 user1 user2` (imprime "user1,user2" si ambos están conectados) o `$0 root` (imprime "root" si está conectado), y notas sobre la necesidad de al menos un parámetro y que solo se mostrarán usuarios conectados, terminando con `exit 1`. Luego, verifica con `if [ $# -lt 1 ]` si se pasó al menos un argumento; si no, muestra "Error: Debes introducir al menos un parámetro" y llama a `mostrar_ayuda`, como se vio en la imagen al ejecutarlo sin parámetros. Si hay argumentos, el script itera sobre `"$@"` usando un bucle `for`, verificando cada usuario con `who | grep -w "$param" > /dev/null` (who lista usuarios conectados y grep -w busca coincidencias exactas); si un usuario está conectado, se añade a la variable `PARAMS`, asignándolo directamente si está vacía o concatenándolo con una coma si ya tiene contenido.

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

Finalmente, si PARAMS está vacía, muestra "Error: Ninguno de los usuarios proporcionados está conectado" y la ayuda, pero si hay usuarios conectados, imprime "Usuarios conectados: \$PARAMS", como en la ejecución `./UsuariosConectados.sh root user2`, que mostró "Usuarios conectados: root" porque root estaba conectado y user2 no. Para usar el script, se crea con nano `UsuariosConectados.sh`, se otorgan permisos con `chmod +x UsuariosConectados.sh`, y se ejecuta desde la terminal, ofreciendo una solución útil para monitorear usuarios conectados con una interfaz clara y amigable.

A continuación, la respectiva imagen:

```

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

alejo@alejo-VirtualBox:~$ nano UsuariosConectados.sh
alejo@alejo-VirtualBox:~$ chmod +x UsuariosConectados.sh
alejo@alejo-VirtualBox:~$ ./UsuariosConectados.sh user1 root user2
Error: Ninguno de los usuarios proporcionados está conectado.
Uso: ./UsuariosConectados.sh usuario1 [usuario2 ...]

Descripción:
Este script toma uno o más nombres de usuarios y verifica si están conectados al sistema.
Los usuarios conectados se imprimen en una sola línea, separados por comas.

Ejemplos:
./UsuariosConectados.sh user1 user2      # Imprime: user1,user2 (si ambos están conectados)
./UsuariosConectados.sh root             # Imprime: root (si root está conectado)

Notas:
- Debes proporcionar al menos un nombre de usuario.
- Solo se mostrarán los usuarios que estén actualmente conectados.
alejo@alejo-VirtualBox:~$

```

7. Script usuarioconectado

El script `usuarioconectado` es una utilidad sencilla en Linux que utiliza Bash para verificar si un único usuario pasado como argumento está conectado al sistema, devolviendo "Sí" si está conectado o "NO" si no lo está, lo que lo hace ideal para ser usado por otros scripts o como herramienta de monitoreo básico.

Comienza con `#!/bin/bash` para ejecutarse en Bash y verifica con `if [ $# -eq 0 ]` si se pasó un argumento; si no, muestra "Error: Debe proporcionar un nombre de usuario como parámetro" y termina con `exit 1`, como se vio en la ejecución `./usuarioconectado` sin parámetros en la imagen. Si se proporciona un argumento, lo asigna a la variable `USUARIO` con `USUARIO=$1` y usa `who | grep -w "$USUARIO" > /dev/null` para comprobar si el usuario está conectado: `who` lista los usuarios activos, `grep -w` busca una coincidencia exacta del nombre, y la salida se redirige a `/dev/null` para ocultar detalles; si hay coincidencia, imprime "Sí", de lo contrario, "NO". Esto se observó al ejecutar `./usuarioconectado juan` (devolviendo "NO" porque juan no estaba conectado) y `./usuarioconectado alejo` (devolviendo "Sí" porque alejo estaba conectado en `tty7` a las 07:03 del 26 de mayo de 2025, según el comando `who`).

Para usar el script, se crea con nano `usuarioconectado`, se otorgan permisos con `chmod +x usuarioconectado`, y se ejecuta desde la terminal con un nombre de usuario, proporcionando una forma directa y eficiente de verificar el estado de conexión de un usuario en el sistema.

A continuación, la respectiva imagen:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

```

alejo@alejo-VirtualBox: ~
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

alejo@alejo-VirtualBox:~$ nano usuarioconectado
alejo@alejo-VirtualBox:~$ chmod +x usuarioconectado
alejo@alejo-VirtualBox:~$ ./usuarioconectado nombre_usuario
NO
alejo@alejo-VirtualBox:~$ ./usuarioconectado juan
NO
alejo@alejo-VirtualBox:~$ who
./usuarioconectado nombre_usuario
alejo    tty7        2025-05-26 07:03 (:0)
alejo@alejo-VirtualBox:~$ ./usuarioconectado alejo
SI
alejo@alejo-VirtualBox:~$

```

8. Script modificar_path.sh

El script modificar_path.sh es una herramienta práctica en Linux que utiliza Bash para añadir una ruta específica al PATH del sistema en el archivo .bashrc, permitiendo ejecutar scripts de esa carpeta desde cualquier ubicación, con verificaciones para evitar errores y duplicados. Comienza con `#!/bin/bash` y define dos variables:

`CARPETA_EJERCICIOS="/ruta_carpeta_ejercicios"` (la ruta a añadir, que debe ajustarse según el caso) y `BASHRC="$HOME/.bashrc"` (la ubicación del archivo de configuración del usuario). Primero, verifica con `if [ ! -d "$CARPETA_EJERCICIOS" ]` si la carpeta existe; si no, muestra "Error: La carpeta \$CARPETA_EJERCICIOS no existe" y termina con `exit 1`, como se vio en la imagen al ejecutar el script con una ruta inexistente.

Luego, usa `grep -Fx "export PATH=\$PATH:$CARPETA_EJERCICIOS" "$BASHRC" > /dev/null` para comprobar si la línea ya está en .bashrc, evitando duplicados; si está, muestra "La ruta ya está en \$BASHRC" y termina con `exit 0`. Si no está duplicada, añade `export PATH=\$PATH:$CARPETA_EJERCICIOS` al final de .bashrc con `>>`, imprime "Ruta \$CARPETA_EJERCICIOS añadida al PATH en \$BASHRC", y sugiere ejecutar `source ~/.bashrc` para aplicar los cambios en la sesión actual. Para usarlo, se crea con `nano` `modificar_path.sh`, se otorgan permisos con `chmod +x modificar_path.sh`, y se ejecuta tras ajustar `CARPETA_EJERCICIOS` a una ruta válida, como `/home/alejo/Ejercicios`, facilitando la gestión de scripts en el entorno del usuario.

A continuación, la respectiva imagen:

```

alejo@alejo-VirtualBox: ~
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

alejo@alejo-VirtualBox:~$ nano modificar_path.sh
alejo@alejo-VirtualBox:~$ nano modificar path.sh
alejo@alejo-VirtualBox:~$ chmod +x modificar_path.sh
alejo@alejo-VirtualBox:~$ ./modificar_path.sh
Ruta /home/alejo/Ejercicios añadida al PATH en /home/alejo/.bashrc
Ejecuta 'source ~/.bashrc' para aplicar los cambios en la sesión actual
alejo@alejo-VirtualBox:~$ source ~/.bashrc
alejo@alejo-VirtualBox:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/home/alejo/Ejercicios
alejo@alejo-VirtualBox:~$

```

9. Script modificar_script_para_llamar_usuarioconectado.sh

El script modificar_script_para_llamar_usuarioconectado.sh es una herramienta útil en Linux que utiliza Bash para modificar automáticamente otro script (`mostrar_parametros.sh`) y añadirle la funcionalidad de verificar si los usuarios pasados como argumentos están conectados, utilizando un script externo llamado `usuarioconectado`, con medidas de seguridad para evitar errores.

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

Comienza con `#!/bin/bash` para ejecutarse en Bash y define la variable `SCRIPT_OBJETIVO="mostrar_parametros.sh"` como el archivo a modificar. Luego, verifica con `if [ ! -f "$SCRIPT_OBJETIVO" ]` si el archivo existe; si no, muestra "Error: El archivo \$SCRIPT_OBJETIVO no existe" y termina con `exit 1`, como se vio en la imagen al ejecutarlo en `/home/alejo/Ejercicios`, donde `mostrar_parametros.sh` no estaba presente. Si el archivo existe, crea una copia de respaldo con `cp "$SCRIPT_OBJETIVO" "${SCRIPT_OBJETIVO}.bak"` para evitar pérdida de datos y, usando `cat << 'EOF' >>`, añade al final de `mostrar_parametros.sh` un bloque de código que imprime una línea en blanco, un mensaje ("Comprobando si los usuarios están conectados:"), y un bucle `for` que itera sobre los argumentos (`"$@"`), ejecuta `./usuarioconectado` para cada uno, guarda el resultado (SÍ o NO) en la variable `estado`, y lo muestra en el formato `usuario → estado`.

Finalmente, confirma la modificación con "Script '`$SCRIPT_OBJETIVO`' modificado exitosamente." y explica que se añadió una llamada a `usuarioconectado` por cada argumento. Para usarlo, se crea con `nano` `modificar_script_para_llamar_usuarioconectado.sh`, se otorgan permisos con `chmod +x`, y se ejecuta en un directorio donde `mostrar_parametros.sh` exista, asegurando una modificación segura y funcional del script objetivo.

A continuación, la respectiva imagen:

```

alejo@alejo-VirtualBox: ~/Ejercicios
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

alejo@alejo-VirtualBox:~$ cd /home/alejo/Ejercicios
alejo@alejo-VirtualBox:~/Ejercicios$ nano modificar_script_para_llamar_usuarioconectado.sh
alejo@alejo-VirtualBox:~/Ejercicios$ chmod +x modificar_script_para_llamar_usuarioconectado.sh
alejo@alejo-VirtualBox:~/Ejercicios$ ./modificar_script_para_llamar_usuarioconectado.sh
Script 'mostrar_parametros.sh' modificado exitosamente.
Se añadió una llamada a 'usuarioconectado' por cada argumento recibido.
alejo@alejo-VirtualBox:~/Ejercicios$

```

Resumen de los scripts

A continuación, brindo el resumen completo de todos los scripts para mayor facilidad en la comprensión lectora:

- **modificar_script_para_llamar_usuarioconectado.sh:** Modifica `mostrar_parametros.sh` para añadirle la verificación de usuarios conectados.
- **UsuariosConectados.sh:** Lista usuarios conectados en una sola línea, con ayuda detallada.
- **HolaMundo.sh:** Script básico que imprime "Hola Mundo".
- **VerificaParametros.sh:** Muestra parámetros uno por línea, con verificación de argumentos.
- **usuarioconectado:** Verifica si un usuario está conectado, devolviendo "SÍ" o "NO".
- `mostrar_parametros.sh:` Muestra parámetros y verifica si son usuarios conectados.
- **ParametrosComasAyuda.sh:** Muestra parámetros separados por comas, con ayuda detallada.
- **modificar_path.sh:** Añade una carpeta al PATH en `.bashrc`.
- **ParametrosComas.sh:** Muestra parámetros separados por comas, sin ayuda detallada.

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	26-05-2025
	Nombre: Alejandro	

Y con este último resumen explicativo de todos los scripts finalizo el desarrollo de este laboratorio. **¡Mil gracias, respetado profesor por estos valiosos conocimientos aportados, de muy valiosa utilidad en mi ámbito profesional!**

CONCLUSIÓN DE LA ACTIVIDAD

Para concluir este trabajo, puedo afirmar que el uso de herramientas como VirtualBox y Linux Mint representa una metodología eficaz para comprender e implementar conceptos clave de los sistemas operativos avanzados. La virtualización no solo permite explorar distintos entornos sin comprometer el sistema anfitrión, sino que también habilita una plataforma segura para experimentar y cometer errores sin consecuencias permanentes.

Durante el desarrollo del laboratorio, pude constatar que la instalación de un sistema operativo en una máquina virtual va más allá de una simple práctica técnica; es una experiencia integral que abarca la asignación eficiente de recursos, el manejo de archivos ISO, la gestión de dispositivos virtuales, y el conocimiento de comandos esenciales para la administración del sistema.

En lo personal, destaco la utilidad de esta actividad como punto de partida para adentrarse en temas más complejos como la gestión de procesos, manejo de memoria, sistemas de archivos y comunicación entre procesos, los cuales son fundamentales en el estudio de la arquitectura y funcionamiento interno de un sistema operativo.

Finalmente, esta experiencia me permitió comprender el valor de la virtualización como herramienta de aprendizaje y práctica profesional, brindando un entorno flexible y replicable donde es posible consolidar habilidades que serán clave en el desarrollo de futuras actividades en el campo de la ingeniería informática. **Muchas gracias respetado y estimado, profesor.**

BIBLIOGRAFÍA

A continuación, la bibliografía implementada en la búsqueda de información:

- ✓ Tema 1. Concepto de sistema operativo. Tema 2. Historia de los sistemas operativos. Tema 3. Estructura de un sistema operativo. Tema 4. Llamadas al sistema. Tema 5. Windows y Linux. Tema 6. El Shell de Linux. Tema 7. Programación Shell,
- ✓ Clases virtuales con el profesor Ing, Laerte Alberto Ferreira Castro.
- ✓ Shotts, W. E. (2019). The Linux Command Line: A Complete Introduction (2nd ed.). No Starch Press. Manual práctico sobre Bash, comandos, scripting y automatización en Linux.
- ✓ Sobell, M. G. (2018). A Practical Guide to Linux Commands, Editors, and Shell Programming (4th ed.). Pearson Education. Detalles técnicos y ejemplos sobre nano, permisos (chmod), y programación de scripts.
- ✓ The Linux Documentation Project. (2024). Advanced Bash-Scripting Guide. Recuperado de: <https://tldp.org/LDP/abs/html/>. Guía avanzada para escribir y modificar scripts Bash.